

Hybrid Mechanical and Agentic Specification Inference

Anonymous Author(s)

Anonymous Institution(s)

Abstract. Formal verification suffers from the cost of specification: to establish high-level facts, contracts have to be written for supporting functions, many of them mechanical and low-level. Specification inference derives such contracts from the program. We present a hybrid agentic-mechanical inference approach for the Move smart contract language, which has many similarities with Rust, and is integrated with the Move Prover. An agent controls inference and verification via a tool which offers, beside calling the Move Prover, deriving specs from code via weakest precondition analysis. Equipped with this tool, and loop invariants guessed by AI, the agent is capable to derive complete specs for non-trivial problems. We evaluate the approach on 20 verification targets extracted from a private production codebase, with completeness checked by withheld mutants, under a frontier model, Sol 5.6, and a mid-tier model of the same family, Terra 5.6. The unaided agent completes 93% of its cells. With WP, both models complete more, and the mid-tier model completes as many as the frontier model at 55% of its cost.

Keywords: Specification inference · Weakest precondition · Reference elimination · LLM agents · Move Prover

1 Introduction

A roadblock in program verification adoption is the *cost of specification*. The properties developers care about are high-level — a call chain aborts only under stated preconditions, an entrypoint satisfies its postconditions, stored data respects an invariant — but to prove any of them, the supporting functions need contracts of their own, and those are often boilerplate, as they restate what the code does. *Specification inference* derives such contracts from the program, so that effort can go to the properties worth stating by hand. The classical engine for mechanical inference is Dijkstra’s weakest-precondition transformation (WP) [9]: propagated backward through a function, it yields the exact abort condition and the exact postcondition. However, it depends on loop invariants to provide an induction hypothesis [7]. Invariant synthesis has no complete technique despite a long history [8] of research.

This paper describes a novel combination of agentic specification inference — researched e.g. in [?] — and weakest precondition transformation. The agent conducts the inference tactic, supplies loop invariants, and calls WP and verification

as a tool to aide the work. The approach is implemented for the Move smart contract language which shares many similarities with Rust. Move is equipped with strong verification support by the Move Prover (MVP) [10,13]. Move shares the reference logic with safe Rust, and is therefore amenable to reference elimination, which has been applied in the Move Prover since its incubation [10]. On reference-eliminated Move IR, we run classical WP, mapping Moves reference logic into the predicate calculus of the spec language. Moreover, via the concept of behavioral predicates [?], which allow to reason about the specification of others functions as a black box, WP becomes fully modular.

The combination is evaluated on a corpus of 20 actual inference problems drawn from a private codebase (that of a production perpetual trading system) — private so that contracts must be inferred rather than recalled — with three tactics, one without the WP tool and two with, under two models of one family, the frontier model Sol 5.6 and the mid-tier model Terra 5.6, and with completeness checked by withheld mutants. Without the WP tool the agent already does well: it produced a complete specification in 93% of its cells under both models, including on targets that require inventing lemmas and recursive helper functions. With WP, both models complete more cells, and the remaining failures occur on one target whose contract needs a permutation argument that WP cannot supply either. The effect on cost depends on the model. For the frontier model, the hybrid tactics cost about the same as the agent alone, since the model finds the invariants unaided and then carries the derived contract in its context. For the mid-tier model, they cost 20% less, and with the tool it completes as many cells as the frontier model at 55% of its cost.

Contributions. The paper presents a novel combination of agentic and mechanic specification inference, integrated with the Move Prover, with a detailed empirical evaluation on a set of real-world code examples, demonstrating that combinations like this can lower the cost and increase reliability in this domain. As a side-effect, we demonstrate that reference elimination in the style of the Move Prover is a good starting point for specification inference.

The Move Prover was developed for the Diem project and is since maintained by the blockchain company behind it; it has verified Diem’s payment system and critical components of a public blockchain [19,3]. Move shares Rust’s safe references, linear types, algebraic data types, and higher-order functions [4]; the mechanical half of our approach relies on exactly these traits, so we expect it to carry over to verifiers for Rust-like languages that eliminate references the same way.

2 Hybrid Inference by Example

Hybrid inference combines two sources of specifications: the mechanical WP analysis of Sec. 3, which is exact where it applies and stops at loops, and a coding agent, which proposes what the analysis cannot derive and repairs against verifier feedback. The prover is connected to the agent through an MCP service that

offers a tool for verification and a tool for WP inference, and the agent carries a set of skills describing the language, the tools, and the workflow (Sec. 4). We walk through one session; the example is written in Move but should be readable without knowing the language [4].

The example we look at is an iterative computation of the exponential, b^e :

```

user
fun pow(base: u64, exp: u64): u64 {
  let result = 1;
  let i = 0;
  while (i < exp) {
    result = result * base;
    i = i + 1;
  }
  result
}

```

Based on the instructions, the first thing the agent does is call WP. The answer points to the missing loop invariant, including unrolling the loop symbolically and showing the values of the affected loop variables at each entry:

```

wp tool
WP inferred 'vacuous' conditions after this loop without an
invariant. ..
--> sources/pow.move:5:14
|
5 |      while (i < exp) {
|      ^
|
= loop-invariant evidence
= bounded loop-head facts :
  head[0]: head[0].i = 0
           head[0].result = 1
  head[1]: 0 < exp ==> head[1].i = 1
           0 < exp ==> head[1].result = base
  head[2]: 1 < exp ==> head[2].i = 2
           1 < exp ==> head[2].result = base * base
  ..

```

Based on the diagnosis, the agent next derives the loop invariant. A recursive specification helper function is introduced for accumulating the result. Before returning the result, the agent uses the verification tool to check whether its solution verifies:

```

agent
fun pow(base: u64, exp: u64): u64 {
  ..
  while (i < exp) {
    result = result * base;
  }
}

```

```

    i = i + 1;
  } spec {
    invariant [inferred] i ≤ exp;
    invariant [inferred] result = math_pow(base, i);
  }
  ..
}
// specification helper function
spec fun math_pow(base: num, exp: num): num {
  if (exp ≤ 0) { 1 }
  else { math_pow(base, exp - 1) * base }
}

```

In the next turn the agent runs the WP MCP tool again. With the loop invariant present, the complete specification is derived. However, the aborts condition is marked as `sathard` by the WP tool. The marker is set when a derived condition is exact but its satisfiability is expected to be hard for the solver — here because of the existential quantification, which says that the computation will abort if for some `y` the exponent multiplication overflows. Note that the Move Prover uses unbounded integers, `num`, in the specification language, making it possible to reason about overflow in the programming language’s `u64`:

```

spec pow { // post and aborts condition for pow
  ensures [inferred] result = math_pow(base, exp);
  aborts_if [inferred = sathard] ∃ y: u64:
    math_pow(base, y) * base > MAX_U64
    && (∀ x: num: 0 ≤ x && x ≤ y
      ⇒ math_pow(base, x) ≤ MAX_U64)
    && y < exp
}

```

wp tool

The agent tries to verify this specification, but it times out with the following diagnosis:

```

verification failed:
error: verification out of resources/timeout
      (timeout set to 5s)
= timeout analysis replay:
  mixed quantifier and nonlinear arithmetic
= top quantifier activity:
  quantifier at sources/pow.move:24
  -- 55,793+ observed instantiations

```

verify tool

The quantifier the diagnosis is pointing to is the defining recursive axiom of the specification helper function `math_pow`, which is apparently endlessly unfolded in order to prove the `sathard` aborts condition. In the next turn, the agent

introduces lemmas to characterize the monotonicity of multiplication, which allows it to rewrite the aborts condition:

```

spec pow {
  pragma opaque;
  ensures [inferred] result = math_pow(base, exp);
  /// The partial products are non-decreasing, so the
  /// loop overflows exactly when its last product does.
  aborts_if [inferred]
    exp > 0 && math_pow(base, exp - 1) * base > MAX_U64;
} proof {
  /// Bounds every iteration's product by the last one;
  /// triggers on the invariant's 'math_pow(base, i)'.
  ∀ k: num {math_pow(base, k)}
    apply math_pow_last_is_max(base, k, exp);
}
spec lemma math_pow_last_is_max(
  base: num, k: num, n: num) {
  requires base ≥ 0 && 0 ≤ k && k < n;
  ensures math_pow(base, k) * base
    ≤ math_pow(base, n - 1) * base;
} proof {
  apply math_pow_mono(base, k + 1, n);
}
spec lemma math_pow_mono ... // left out for brevity

```

3 Weakest Preconditions over Eliminated References

The mechanical half of the system is a weakest-precondition analysis [9] that reads off abort conditions, modifies clauses, and consequence-style postconditions from a function's bytecode. WP itself is textbook. Two things make it applicable to a language with Rust-style references and make its output readable: it runs on MVP's reference-eliminated bytecode, where backward predicate propagation is plain substitution, and a reverse mapping carries the derived predicates back to the references of the source. This section describes both, and then what the elimination buys compared to reasoning over references directly.

3.1 Reference Elimination

Move has Rust-style references: $\&T$ for shared reads and $\&\text{mut } T$ for exclusive writes, both with a stack-bounded lifetime, and the borrow checker guarantees that a live $\&\text{mut}$ is the only access path to its referent. MVP exploits this guarantee to rewrite bytecode into a reference-free form [10], originally in order to keep the verification conditions first-order and free of a heap model. Two passes perform the elimination. Shared references are inlined: $\&T$ disappears and reads through it become reads of the underlying value. A mutable reference becomes a pair

of the value it currently denotes and a *borrow path* back to its root — a local, a `&mut` parameter, or a global resource $R[a]$ — and every mutation is followed by an explicit *write-back* along that path, inserted by a borrow analysis, which rebuilds the parent value with the field or element replaced. A `&mut` parameter becomes an in/out value parameter: it enters as a plain value and leaves as the value written back last. Where the root of a reference is not statically unique, as in `if (c) &mut x else &mut y`, the write-back is guarded by an explicit test of which parent the reference was taken from. Before WP runs, the bytecode further passes through MVP’s specification instrumentation: each function-level `requires`, `aborts_if`, `ensures`, or `modifies` clause and each loop `invariant` becomes an `assume` or `assert` at the appropriate program point, and calls are summarized by the *behavioral predicates* of [13]: `requires_of<f>`, `aborts_of<f>`, `ensures_of<f>`, `modifies_of<f>`, and `result_of<f>` reify a callee’s contract as predicates over the call site’s arguments and result. At the input to WP, the bytecode is a sequence of assignments over plain values interleaved with `assume` and `assert`.

3.2 The Predicate Transformer

The analysis is a backward dataflow pass over the control-flow graph. Its state at a program point is a triple (E, A, ℓ) : a set E of *ensures* conjuncts that hold on every normal return reached from the point, a set A of *abort* disjuncts under which the function aborts, and a *memory label* ℓ naming the memory version the conditions refer to, together with bookkeeping for the `&mut` parameters and global resources that have been written on the way. At a return, $E = \{\mathbf{result}_i = t_i\}$ for the returned temporaries and $A = \emptyset$; at an `abort`, $A = \{\mathbf{true}\}$. Instructions transform the state as follows.

- *Assignment and operators.* $t := e$ substitutes e for t in E and A ; an operator that can abort — arithmetic overflow and underflow, division by zero, an index out of range, a cast that truncates, a missing enum variant — also adds its abort predicate to A .
- *References.* A borrow $r := \&x$ or $r := \&x.f$ substitutes x or $x.f$ for r ; a write-back along a field edge replaces the parent x by x with field f set to the value written; a global borrow $r := \&\text{mut } R[a]$ substitutes $R[a]$ for r and adds `!exists<R>(a)` to A . Since every reference is a path to a named root, no instruction needs a heap.
- *Calls.* A call $t := f(\bar{a})$ substitutes `result_of<f>( $\bar{a}$ )` for t — or the callee’s specification function, if f is pure — adds `aborts_of<f>( $\bar{a}$ )` to A , and relates the memory before and after the call through `ensures_of<f>`; when the callee has no abort contract, the abort condition is recorded as *partial* rather than invented. Calls are thus summarized modularly: the caller’s contract says what f returns without committing to how.
- *Branches.* At a branch on c the two incoming states are merged path-sensitively: conditions common to both sides are kept as they are, the others are guarded, $c \Rightarrow e$ and $\neg c \Rightarrow e'$ in E , $c \wedge a$ and $\neg c \wedge a'$ in A .

- *Specifications.* An `assert` φ adds φ to E and $\neg\varphi$ to A ; an `assume` φ adds φ as an antecedent to both. This is how the contracts of callees, user-written conditions, and loop invariants enter the derivation.
- *Loops.* Loops are broken at their headers by the cut-point encoding of Barnett and Leino [7]: the back-edge is severed, the loop `invariant` is asserted before and after each iteration, and the loop-modified locations are havoced in between. WP of `havoc x` universally quantifies x in E and existentially in A , so variables that the invariant does not constrain appear quantified in the contract.

The pass iterates to a fixpoint in reverse topological order of the acyclic graph that remains, and the state at entry is the raw contract. Post-processing then strips the labels of the entry and exit memories, resolves the parent tests of non-unique write-backs into path conditions computed from the dominator tree, adds `p = old(p)` for every `&mut` parameter never written, substitutes the global resources captured through borrows, and simplifies: constant folding, boolean and arithmetic identities, tautology removal, and scoping of quantifiers to the conjuncts that mention their variable. Simplification matters more here than in a verifier, where the WP result goes to the solver and is never shown: the raw predicate of a twenty-line function can have hundreds of terms, and the agent and the user see the simplified one.

3.3 Carrying the Result Back

The predicate at entry is stated over the eliminated form, whose in/out parameters and rebuilt parent values have no counterpart in the source signature; the contract the user reads must be stated over the original references. Three cases arise, and each has a source-level spelling. A `&T` parameter needs nothing: the specification language already reads a shared reference as its value. For a `&mut T` parameter `p`, the in/out parameter’s exit value becomes `ensures p = e`, where `e` is the value written back last on the path to the return, and any read of `p` before that write resolves to `old(p)`, the value at entry; an earlier write is folded in by substituting it for `old(p)` in what was already derived, so a sequence of writes chains into one equation. For a global resource, a borrow of `R[a]` that is written back yields `modifies R[a]` together with an equation on the resource’s post-state value, with `old(R[a])` for its value at entry. Consider the transfer of an amount between two balances:

```

struct Balance has key { v: u64 }
fun transfer(from: address, to: address, amount: u64) {
  Balance[from].v -= amount;
  Balance[to].v += amount;
}

```

user

Both statements borrow a resource mutably, read and write a field through the reference, and write back. The derived contract is stated over the resources themselves, and the two borrow roots may coincide, so WP keeps them apart by a case split on `from = to`:

```

spec transfer {
  pragma opaque;
  modifies Balance[from];
  modifies Balance[to];
  aborts_if [inferred] ! exists<Balance>(from);
  aborts_if [inferred] ! exists<Balance>(to);
  aborts_if [inferred] Balance[from].v < amount;
  aborts_if [inferred]
    from ≠ to && Balance[to].v + amount > MAX_U64;
  ensures [inferred]
    from = to ⇒ Balance[from] = old(Balance[from]);
  ensures [inferred] from ≠ to
    ⇒ Balance[from].v = old(Balance[from]).v - amount;
  ensures [inferred] from ≠ to
    ⇒ Balance[to].v = old(Balance[to]).v + amount;
}

```

wp tool

When a function updates global memory in several steps, an equation on the final memory is not enough: the abort condition of the second `move_to` below depends on the memory after the first. The memory labels of the transformer therefore surface as *state labels* naming the intermediate memory states at which a global update happens, and the specification language lets a condition be evaluated at a labeled state, $S \models e$, or over a range of states, $S1..S2 \models e$:

```

fun split_balance(
  dst1: &signer, dst2: &signer, src: address): u64 {
  let Balance { v } = move_from<Balance>(src);
  let half = v / 2;
  move_to(dst1, Balance { v: half });
  move_to(dst2, Balance { v: v - half });
  half
}

```

user

```

spec split_balance {
  pragma opaque;
  modifies Balance[src];
  modifies Balance[signer::address_of(dst1)];
  modifies Balance[signer::address_of(dst2)];
  ensures [inferred] result = old(Balance[src]).v / 2;
  ensures [inferred] ..S1 ⊨ remove<Balance>(src);
  ensures [inferred] S1..S2 ⊨
    publish<Balance>(signer::address_of(dst1),
      Balance { v: old(Balance[src]).v / 2 });
  ensures [inferred] S2.. ⊨
    publish<Balance>(signer::address_of(dst2),
      Balance { v: old(Balance[src]).v

```

wp tool

```

        - old(Balance[src]).v / 2 });
  aborts_if [inferred] ! exists< Balance>(src);
  aborts_if [inferred]
    S1 |~ exists< Balance>(signer::address_of(dst1));
  aborts_if [inferred]
    S2 |~ exists< Balance>(signer::address_of(dst2));
}

```

S1 names the memory between the `move_from` and the first `move_to`, S2 the memory between the two `move_to`s; `publish<R>(a, x)` states that resource `R` at `a` equals `x` over the range, `remove<R>(a)` that none is present, and `exists<R>(a)` is the usual existence predicate. Without the labels, the memory versions the eliminated form threads through the function could be read back only as a single, weaker equation on the final memory.

3.4 Diagnostics for the Agent

WP is complete only where its inputs are: a loop without an invariant and a callee without an abort contract both leave holes, and the tool reports both instead of silently emitting an incomplete contract. After a havoc loop, a condition that quantifies over everything the loop touched carries no information; such *vacuous* conditions are dropped and the loop is reported with *loop-invariant evidence* — the facts that hold at the first few loop heads on each path, computed by bounded unrolling of the same transformer, as in the `head[k]` listing of Sec. 2. A condition that is exact but existentially quantified, typically an abort condition that says *some* iteration overflows, is kept and marked *sathard*, since its satisfiability will be expensive for the solver. An abort condition that could not account for a callee is marked *partial*. Every emitted condition carries an `[inferred]` marker, so that re-runs replace their own output and never disturb user-written specifications, and every function that received a contract is made *opaque*, so that callers reason through it. The agent’s skills are written against these markers: they identify the loop that needs an invariant, the condition to simplify, and the callee to specify first.

3.5 What the Elimination Buys

Reference elimination was introduced to make *verification* of Move tractable [10]: without a heap, the verification conditions stay first-order and the solver needs no frame reasoning. Running WP on the eliminated form is a second use of the same transformation that, to our knowledge, has not been described before, and it avoids the following problems. A predicate transformer over programs with references has to carry a model of locations: `*r` means the contents of whatever `r` points to, an assignment through one reference may change the value read through another, and the transformer needs either an explicit heap with separation or ownership reasoning to keep the substitutions sound [18,14]. Its output is then stated over that model — heap fragments, prophecy variables,

ownership predicates — and has to be translated before a user can read it. On the eliminated form none of this arises. Every reference is a path to a named root, so substitution is exact by construction; the only aliasing that survives is between global resources whose addresses may coincide, and it surfaces as an explicit condition such as `from = to` above. The derived predicates mention only parameters, results, fields, and resources, which is the vocabulary of the specification language, so the reverse mapping of Sec. 3.3 reduces to renaming. And because inference and verification run on the same pipeline, every inferred contract is checked under the semantics that produced it; the instrumented contracts of callees and the loop invariants are available in-line to both. The whole analysis is about nine thousand lines of Rust on top of the existing pipeline, and adds no new semantics to the prover. Whether the same holds for a prophecy-based elimination, which is designed for MVP but not implemented, is open; Sec. 6 returns to it.

4 The Agent Side

The agentic side of the workflow is delivered as a plugin to a general-purpose coding agent: skills that instruct it, hooks that check generated Move as it is written, and the Move toolchain exposed as tools it can call. The setup is at [2]; we summarize the design.

Inference tactics. Inference runs in one of three *tactics*, which differ in whether the WP tool is available and who decides when to use it.

- *agent-only*: WP is unavailable. The contract and any loop invariants are derived directly from the implementation and the contracts of its dependencies, checked as the work proceeds.
- *hybrid-guided*: the workflow of Sec. 2, prescribed as a fixed order — run WP over the scope, take one warned function and write an invariant from the loop-invariant evidence (Sec. 3.4), rerun WP on that function alone until the warning is gone, simplify what WP derived, then check.
- *hybrid-flexible*: WP is available as one inference pass among others, and the agent decides whether and when to call it and how to combine it with its own reasoning.

Skills. A tactic selects the *plan* — the sequence of tasks and which of them consult WP — and nothing else. All other guidance is shared reference material, identical in all three tactics: the specification language, the rules for writing and editing specs, the verification sub-workflow, and the toolchain’s capabilities and limits. Only the description of the WP tool itself is withheld where the tool is absent; the underlying idea of backward reasoning, and what a loop without an invariant costs, is background every tactic receives.

Three areas of that guidance carry most of the weight.

- *Loop invariants*: when to introduce recursive helper functions for iterative quantities, and when to prefer state-level invariants on resources — assumed by the prover at every call site, including each loop iteration — over per-loop inductive arguments.
- *Simplification*: discard vacuous conditions; replace unbounded quantifiers, a frequent source of SMT timeouts, with bounded or closed-form equivalents; and require trigger annotations on the surviving ones.
- *Timeout resolution*: a hierarchy of escalations — strengthen state-level invariants, then introduce spec helpers, then lemmas, then explicit proof scripts — with a strict prohibition on weakening abort or postconditions to silence a timeout, since this gives up a real property only to make the proof succeed.

5 Evaluation

The evaluation asks what the mechanical component adds to an agent, in completeness and in cost, and whether it changes which model is needed. *Arm against arm* (Sec. 5.2) holds the model fixed and varies the tactic: *agent-only*, *hybrid-guided*, and *hybrid-flexible*, run under the same conditions. *Model against model* (Sec. 5.3) repeats the design under a frontier model and a mid-tier model of the same family and asks whether, with WP, the cheaper model reaches the results of the frontier model. Both rounds share the corpus, the protocol, the scoring, and the cost accounting described first.

5.1 Setup

Corpus. The corpus is drawn from the private Move code of a production perpetual trading system: 23 targets from it, two from a public Move library that carry no upstream specification, and two written for the corpus. A private codebase matters here: the model cannot have seen these functions, or any specification of them, during training, so a contract has to be inferred rather than recalled. Every candidate is classified by the features (strata) its contract has to account for — loops and accumulators, early returns and breaks, arithmetic overflow and underflow, casts and truncation, mutable references and in-place permutation, function values, quantifiers, composition through a callee’s contract, and others, 38 in all. A target is admitted only if it proves within seconds against a hand-written reference specification, and before the round three *mutants* are authored per target — patches to the implementation, one per obligation the contract has to pin — each shown to be killed by the reference. Twenty targets are selected to cover all 38 features across 15 modules, 10 of them with loops; three are deliberately guessable as controls, and five near-duplicates are held back. With 3 arms and 4 replicates, a model round has **240 cells**.

Arms and models. The three tactics of Sec. 4 run as arms that differ only in the rendered plugin: the reference material is byte-identical, and in agent-only

the WP tool is absent rather than discouraged. Two rounds use two models of one family through the same runtime, the Codex CLI at high effort: Sol 5.6, the frontier model, and Terra 5.6, the mid-tier model priced at about half of Sol’s list. Each session has a budget of 100–180k output tokens, one hour of wall time, and 40 seconds of solver time per verification condition.

Scoring. A cell reaches *operational success* when a judge outside the agent’s workspace confirms that the package compiles, the requested scope verifies, no weakening construct is present — `pragma verify = false`, partial abort contracts, vacuous postconditions, invented preconditions — the bytecode is unchanged, and the required contract categories are present. The agent’s own claim is never a score, and the agent receives only this acceptance feedback: the mutants are withheld until generation has ended, and a contract is never handed back for strengthening. Operational success is necessary but weak: a vaguer contract passes it more easily. Completeness — postconditions as strong as the behavior — has no deductive check in general; WP guarantees it only given complete loop invariants and callee contracts, and warns when it cannot, a mechanism the agent-only arm lacks. The second level is therefore *mutation adequacy*: after the round, each withheld mutant is applied to the implementation and the prover rerun against the inferred specification. A cell is *complete* when it killed every mutant and *failed* when one survived it; a cell whose scoring stayed inconclusive after one infrastructure retry is *unmeasured* and counts in neither. Every one of the 480 cells reached operational success, so the outcome of a cell is decided by the withheld mutants alone.

Cost. Costs are in US dollars at each model’s published list prices, recomputed from the recorded token counters of every model request rather than taken from the harness’s estimate; startup requests that do no task work are excluded. Per million tokens, Sol 5.6 is \$4 uncached input, \$0.40 cached input, and \$20 output; Terra 5.6 is \$2, \$0.20, and \$12. Thinking counts as output for both. Means are task-weighted, and the 95% intervals are percentile bootstraps that hold the 20 tasks fixed and resample the four replicate blocks within each task with the three arms kept together (100,000 draws); they describe this corpus, not a population of tasks.

5.2 Arm against Arm

Table 1 summarizes both rounds, Fig. 1 shows every cell, and Tables 2 and 3 in the appendix give every target. Under both models the hybrids complete more cells than agent-only: 77 and 77 against 74 under Sol 5.6, and 79 and 78 against 74 under Terra 5.6. All 16 failures occur on two targets. QP-part-025, the in-place partition, is the one target whose contract has to characterize a whole data structure — that the result is a rearrangement of the input — and it defeats every arm under both models, hybrid-guided least under Terra 5.6 (1 cell) and most under Sol 5.6 (3 cells). OV-order-006 is the control that never aborts,

Table 1. Both rounds, per arm: mean cost in US dollars per cell, the hybrid/agent-only cost ratio with its 95% bootstrap interval, cells that killed every withheld mutant, cells a mutant survived, cells whose scoring stayed inconclusive, mean input/output tokens in thousands, and mean wall time per cell in seconds.

model	arm	cost	vs. agent	[95% CI]	complete	failed	unmeas.	I/O k	wall s
Sol 5.6	agent-only	\$0.333	—		74/80	4	2	250/4.9	119
	hybrid-guided	\$0.343	1.03	[0.94, 1.14]	77/80	3	0	263/5.3	125
	hybrid-flexible	\$0.362	1.09	[1.00, 1.18]	77/80	1	2	292/5.1	120
Terra 5.6	agent-only	\$0.235	—		74/80	5	1	372/6.4	124
	hybrid-guided	\$0.188	0.80	[0.73, 0.87]	79/80	1	0	281/5.0	102
	hybrid-flexible	\$0.207	0.88	[0.78, 0.99]	78/80	2	0	342/5.2	103

because its $\&\&$ short-circuits past the guards that would; agent-only stated an abort condition anyway in all four Sol 5.6 cells and two Terra 5.6 cells, and no hybrid cell missed it, since WP derives the absence of an abort exactly.

The effect on cost differs between the models. Under Sol 5.6, hybrid-guided costs $1.03\times$ agent-only [0.94, 1.14] and hybrid-flexible $1.09\times$ [1.00, 1.18] in the task-weighted mean; both intervals reach parity, and the hybrids are cheaper on only 8 and 9 of the 20 targets. The hybrids are cheaper on the small arithmetic contracts — VS-contrib-003, VS-shares-002, PM-curve-027 at about $0.6\times$ — where WP’s contract is already complete and the agent only has to simplify it, and more expensive on the targets that need invention beyond the loop invariant: SM-select-022 at $1.75\times$ under hybrid-guided, whose contract needs a recursive specification function over a function value, and the quantified-abort target TL-lev-020 at $1.3\text{--}1.6\times$. On those targets the derived WP output is large and is re-read on every turn. The hybrids process a twentieth to a sixth more input per cell (263k and 292k against 250k) at similar output, and this additional input, billed at cache-read prices, accounts for the difference.

Under Terra 5.6 the hybrids are cheaper: hybrid-guided costs $0.80\times$ agent-only [0.73, 0.87] and hybrid-flexible $0.88\times$ [0.78, 0.99], cheaper on 14 of the 20 targets each, and both hybrids also finish faster (102 and 103 seconds against 124). The direction of the token counts reverses: the hybrids read *less* than agent-only (281k and 342k against 372k) and write less (5.0k and 5.2k against 6.4k), because the unaided model searches — more verification calls, more repair rounds — where the hybrids start from an exact contract. The most expensive hybrid targets are the same three as under Sol 5.6, SM-select-022, QP-part-025, and TL-lev-020, at $1.2\text{--}1.3\times$.

5.3 Model against Model

Fig. 1 puts the two models side by side within each row, and the comparison is uniform: Terra 5.6 is cheaper than Sol 5.6 on 19 of the 20 targets in every arm, by $0.71\times$ under agent-only and $0.55\times$ and $0.57\times$ under the hybrids in the task-weighted mean, at 231 against 228 complete cells over the round. The exception is BA-base-012, the lemma target, where Terra 5.6 costs $1.2\text{--}1.5\times$

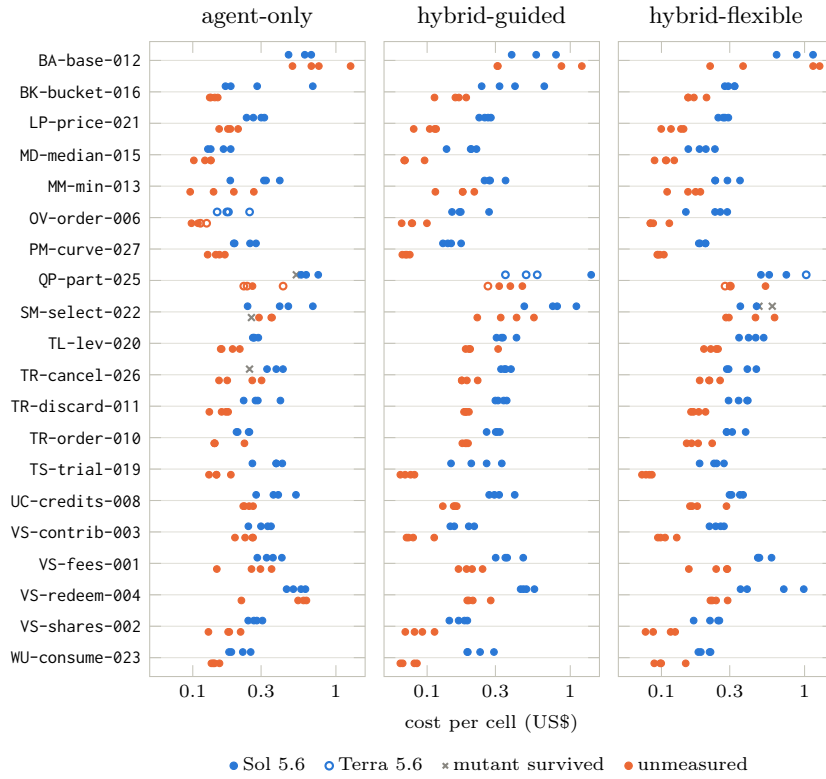


Fig. 1. Every cell of both rounds: cost in US dollars on a log scale, one panel per tactic, one row per target, and within each row the four Sol 5.6 replicates above the four Terra 5.6 replicates. A filled dot is a complete cell, a hollow one a cell a withheld mutant survived, a cross a cell whose scoring stayed inconclusive.

Sol 5.6 because it needs more attempts to find the monotonicity lemma. Since Terra’s price list is about half of Sol’s, the agent-only ratio of 0.71 implies that Terra spends about 40% *more* tokens per cell to reach the same result, while the hybrid ratios near 0.55 imply that with WP it spends about the same: the tool closes the token gap between the two models. In the figure, the replicate spread is about the same for both models in every arm, a factor of about 1.6 between the cheapest and the most expensive replicate of a typical target. What changes with the tactic is how far the Terra replicates lie to the left of the Sol replicates: little under agent-only, and clearly on nearly every row under hybrid-guided.

Together, the two rounds show two effects of WP. The first is on completeness and does not depend on the model: under both, the hybrids complete more cells, never miss the control whose abort condition WP derives exactly, and fail only on the one target whose contract needs a permutation argument that WP cannot supply either. The second is on cost and depends on the model. WP supplies exact

contracts, but its output also enlarges the context the model re-reads on every turn. Sol 5.6 finds the invariants unaided, so it pays for the additional context without saving elsewhere ($1.03\text{--}1.09\times$, parity within the intervals). Terra 5.6 would otherwise search, and the savings outweigh the context ($0.80\times$); with WP, it completes as many cells as Sol 5.6 at 55% of its cost. With four replicates and failures concentrated on two targets, the completeness differences are descriptive; the Terra 5.6 cost intervals exclude parity, the Sol 5.6 ones include it.

6 Discussion and Conclusion

6.1 Interpreting the Results

The evaluation has three results. An agent with no mechanical help produced a complete specification in 93% of its cells under both models. With the WP tool the rate is 97%, and the remaining failures occur on the one target whose contract has to pin down a data structure. And the effect on cost depends on the model, from parity under Sol 5.6 to $0.8\times$ under Terra 5.6.

Agent-only does surprisingly well. Given no WP tool at all, the direct tactic found the same recursive spec helpers, lemmas, and loop invariants as the hybrids, under both models. Its method was consistent: it wrote its own mutation probes and repaired the contract from the counterexamples. It failed in two ways. On the control that never aborts it stated an abort condition anyway, in six cells under Sol 5.6 and Terra 5.6; WP derives the absence of an abort exactly, and no hybrid cell made that mistake. On the partition it pinned a scalar — where the partition point lands — and left the vector’s contents unconstrained, which a mutant that permutes them exposes; the hybrids, which start from the exact postcondition WP derives, fail there at about the same rate, because the permutation argument is beyond WP as well. Two caveats apply. The targets are individual functions of a few dozen lines, and while several need invention, none requires reasoning across a module or a call graph of any depth. And the hybrids’ cost is highest on exactly the invention-heavy targets, where the mechanically derived contract is large and the model re-reads it on every turn. We expect both effects to grow with the size of the target.

Cheaper models with WP. Most of what WP supplies is a diagnosis — *this loop has no invariant, and here are its first iterations* — plus an exact contract that costs context to carry. A model that finds the invariant unaided pays for the context and gains only completeness: under Sol 5.6 the hybrids cost $1.03\text{--}1.09\times$ the agent alone, with the intervals reaching one. A model that would otherwise search saves more than the context costs: under Terra 5.6 the hybrids cost $0.80\times$ and $0.88\times$, both intervals below one, finish faster, and complete as many cells as Sol 5.6 at 55% of its cost, whereas without the tool Terra spends 40% more tokens than Sol for the same result. On this corpus, WP thus makes the frontier model unnecessary: the exact contract, the named loop, and the loop-head facts

are what the smaller model cannot reliably reconstruct on its own, and WP supplies them cheaply. We expect this to extend to models that are cheaper still. The one cost WP adds is context: its output is re-read on every turn, and at API prices this accounts for the hybrids’ additional cost under Sol 5.6. For a local model that cost is latency rather than money, and a model specialized to this workflow, trained to read WP output directly, should need fewer rounds with the tool. The completeness effect does not depend on the model: the failures WP removes — the vacuous postcondition after a havocked loop, the invented abort condition — occur under both models.

Carrying over to prophecies. The reverse mapping of Sec. 3.3 relies on the elimination being *path-based*: an eliminated reference is still a path to a location the source can name, so the derived predicate has a source-level spelling. A *prophecy*-based elimination in the style of RustHorn and Creusot [18], in which a mutable reference is a pair of its current value and a prophecy variable for its final value, pinned when the borrow ends, has been designed for MVP but not implemented; it would need no borrow paths and would handle borrows through function values, which the path-based model cannot. A contract is model-agnostic, so inference could keep running on the path-based form regardless. Whether WP could run on the prophecy form itself and still be carried back is open. For a `&mut` parameter the prophecy *is* the post-state value and has the spelling `p`; for a reborrow through a function value the final value of the borrow has no source-level counterpart at all, and a predicate over it would have to be existentially closed or restated through the callee’s contract. We do not know yet whether that always yields a readable contract.

Open questions. The evaluation leaves the questions of scale open. Its targets are functions, not modules; its corpus is one codebase in one domain; its two models are of one family; and its cost measure is dollars under their price schedules. We plan to apply inference retrospectively to a public blockchain framework, which already carries hand-written specifications, and to compare the inferred contracts against them — a comparison this round cannot make, since its targets deliberately have no specification of their own.

6.2 Related Work

Specification inference predates large language models — Daikon [11] learned likely invariants from program traces — but the techniques that followed stayed tied to particular property classes and source languages, and the advent of LLMs has revived interest in the area. Pei et al. [20] showed that fine-tuned LLMs can predict program invariants statically at quality competitive with dynamic analysis. Kamath et al. [15] pair an LLM that *proposes* loop invariants with a symbolic checker that *validates* them, retrying on failure — a propose-and-validate loop that recurs in much of the subsequent work. For Verus, AlphaVerus [1] bootstraps verified code by self-improving translation with tree search over verifier feedback,

and VeriStruct [21] synthesizes Verus-style specifications and proofs for Rust functions, modules, and data structures through planner-orchestrated phases. For smart contracts in Solidity, PropertyGPT [17] retrieves human-written Certora properties as input examples for specification generation. Most directly related to our work, MSG [12] targets the same Move specification language through a skill-based agentic design that consumes verifier feedback beyond a binary success/failure verdict. Across these systems the LLM remains responsible for the entire specification; the verifier serves as a binary oracle or, in MSG and our setup, as a feedback channel, but never as a contributor of mechanically derived content. A separate line of work, exemplified by DafnyPro [5] and AutoVerus [22], tackles the complementary problem of synthesizing the auxiliary proof annotations a verifier needs when the specification is already given. Our plugin can also synthesize proof hints (Sec. 2), but this is not the focus of this paper.

Reference elimination, the transformation underlying our WP analysis, was introduced in [10], and equivalents exist for Rust. Aeneas [14] compiles Rust to a pure functional model in a target prover (Lean, F*, Coq, HOL4) via forward and backward functions, while RustHorn [18] encodes a mutable borrow as a pair of its current value and a prophecy for its value at the end of the borrow, reducing verification to constrained Horn clauses. Neither infers specifications — in Aeneas the user writes them in the target prover, against the translated model — and both consume the reference-free form only in the forward direction. Our use of the transformation is the opposite: WP runs on the reference-free form, and the conditions it derives are carried *back* through the elimination. To our knowledge this inversion has not been described before. Our backend IVL, Boogie [6,16], also performs WP on the procedural code passed in [7] and passes the result on to the SMT solver; the difference is that Boogie’s WP is not intended for external consumption, whereas ours has to be read, which is reflected in the reverse mapping and in the effort spent on simplification.

Two aspects thus distinguish our setup from the work above. First, to our knowledge, no prior LLM-based spec-inference system uses a sound mechanical baseline alongside the LLM. WP itself produces a substantial part of the inferred specification — abort conditions, modifies clauses, and consequence-style postconditions — so the AI is left only with what WP cannot recover, and the two streams are validated together by the Move Prover. Second, the workflow is delivered as a plugin to a general-purpose, interactive coding agent via skills, edit-time hooks, and an MCP server, rather than as a custom-built tool: the agent participates in the loop and the user can intervene at any point.

6.3 Conclusion

We have presented a specification inference for the Move Prover that runs WP on reference-eliminated bytecode, where backward propagation is substitution and the derived contracts can be mapped back to the references of the source, and we have combined it with a coding agent that supplies what WP cannot: loop invariants, simplification, and proof hints. The prover checks both. Reference elimination was introduced for verification; this work shows that it serves inference

as well, and the approach should carry over to verifiers for Rust-like languages that eliminate references along paths. A controlled comparison on 20 targets drawn from a private perpetual-trading codebase — three tactics, four replicates each, two models — found that a pure agent produces a complete specification in 93% of its cells, even where lemmas and helper functions have to be invented; that with WP more cells are complete under both models, the failures that remain confined to the one target whose contract has to pin down a data structure; and that the hybrids cost about the same as the agent alone under the frontier model and 20% less under the mid-tier model, which then matches the frontier model’s completeness at 55% of its cost. With WP, a cheaper model suffices, and we expect local and specialized models, for which the additional context costs least, to benefit further.

Data Availability Statement. The round archives behind every table and figure — per-cell costs, token counters, wall times, and mutant-gate outcomes — and the scripts that produce them from the archives are published with the inference plugin at [2]. The corpus targets are extracted from a private codebase and are not published; the two targets written for the corpus are included.

References

1. Aggarwal, P., Parno, B., Welleck, S.: AlphaVerus: Bootstrapping formally verified code generation through self-improving translation and tree refinement. In: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025). Proceedings of Machine Learning Research, vol. 267 (2025)
2. Anonymous: Specification-inference plugin for the move prover (2026), source available on request; released with the camera-ready
3. Aptos Labs: The Aptos Framework Book. <https://aptos-labs.github.io/framework-book/> (2023), accessed: 2026-05-09
4. Aptos Labs: The Move on Aptos Book. <https://aptos-labs.github.io/move-book/> (2023), accessed: 2026-05-09
5. Banerjee, D., Bouissou, O., Zetsche, S.: DafnyPro: LLM-assisted automated verification for Dafny programs (2026), <https://arxiv.org/abs/2601.05385>, dafny Workshop, POPL 2026
6. Barnett, M., Chang, B.Y.E., DeLine, R., Jacobs, B., Leino, K.R.M.: Boogie: A modular reusable verifier for object-oriented programs. In: International Symposium on Formal Methods for Components and Objects. pp. 364–387. Springer (2005)
7. Barnett, M., Leino, K.R.M.: Weakest-precondition of unstructured programs. In: Proceedings of the 6th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering. pp. 82–87. Association for Computing Machinery, New York, NY, USA (2005). <https://doi.org/10.1145/1108792.1108813>
8. Cousot, P., Halbwachs, N.: Automatic discovery of linear restraints among variables of a program. In: Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL ’78). pp. 84–96. ACM (1978). <https://doi.org/10.1145/512760.512770>
9. Dijkstra, E.W.: Guarded commands, nondeterminacy and formal derivation of programs. Communications of the ACM **18**(8), 453–457 (1975). <https://doi.org/10.1145/360933.360975>

10. Dill, D.L., Grieskamp, W., Park, J., Qadeer, S., Xu, M., Zhong, J.E.: Fast and reliable formal verification of smart contracts with the move prover. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022). Lecture Notes in Computer Science, vol. 13243, pp. 183–200. Springer (2022). https://doi.org/10.1007/978-3-030-99524-9_10
11. Ernst, M.D., Cockrell, J., Griswold, W.G., Notkin, D.: Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering* **27**(2), 99–123 (2001). <https://doi.org/10.1109/32.908957>
12. Fu, Y.F., Xu, M., Kim, T.: Agentic specification generator for Move programs. In: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025). pp. 1286–1298 (2025). <https://doi.org/10.1109/ASE63991.2025.00110>
13. Grieskamp, W., Zhang, T., Kashyap, V., Silverman, J.: Formal verification of imperative first-class functions in Move. *CoRR* **abs/2605.10007** (2026), <https://arxiv.org/abs/2605.10007>, preprint; to appear in FMCAD 2026
14. Ho, S., Protzenko, J.: Aeneas: Rust verification by functional translation. *Proc. ACM Program. Lang.* **6**(ICFP), 116:1–116:31 (2022). <https://doi.org/10.1145/3547647>
15. Kamath, A., Senthilnathan, A., Chakraborty, S., Deligiannis, P., Lahiri, S.K., Lal, A., Rastogi, A., Roy, S., Sharma, R.: Finding inductive loop invariants using large language models (2023), <https://arxiv.org/abs/2311.07948>
16. Leino, K.R.M., Rümmer, P.: A polymorphic intermediate verification language: Design and logical encoding. In: Esparza, J., Majumdar, R. (eds.) TACAS. pp. 312–327. Springer Berlin Heidelberg, Berlin, Heidelberg (2010)
17. Liu, Y., Xue, Y., Wu, D., Sun, Y., Li, Y., Shi, M., Liu, Y.: PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation. In: Network and Distributed System Security Symposium (NDSS 2025) (2025). <https://doi.org/10.14722/ndss.2025.241357>
18. Matsushita, Y., Tsukada, T., Kobayashi, N.: RustHorn: CHC-based verification for Rust programs. *ACM Trans. Program. Lang. Syst.* **43**(4), 1–54 (2021). <https://doi.org/10.1145/3462205>
19. Park, J., Zhang, T., Grieskamp, W., Xu, M., Giacomo, G.D., Chen, K., Lu, Y., Chen, R.: Securing Aptos Framework with Formal Verification. In: 5th International Workshop on Formal Methods for Blockchains (FMBC 2024). Open Access Series in Informatics (OASIcs), vol. 118, pp. 9:1–9:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2024). <https://doi.org/10.4230/OASIcs.FMBC.2024.9>
20. Pei, K., Bieber, D., Shi, K., Sutton, C., Yin, P.: Can large language models reason about program invariants? In: Proceedings of the 40th International Conference on Machine Learning (ICML 2023). Proceedings of Machine Learning Research, vol. 202, pp. 27496–27520. PMLR (2023), <https://proceedings.mlr.press/v202/pei23a.html>
21. Sun, C., Sun, Y., Amrollahi, D., Zhang, E., Lahiri, S.K., Lu, S., Dill, D.L., Barrett, C.: VeriStruct: AI-assisted automated verification of data-structure modules in Verus. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026). Lecture Notes in Computer Science, Springer (2026), <https://arxiv.org/abs/2510.25015>, arXiv:2510.25015
22. Yang, C., Li, X., Misu, M.R.H., Yao, J., Cui, W., Gong, Y., Hawblitzel, C., Lahiri, S.K., Lorch, J.R., Lu, S., Yang, F., Zhou, Z., Lu, S.: AutoVerus: Automated proof generation for Rust code. *Proc. ACM Program. Lang.* **9**(OOPSLA2) (2025). <https://doi.org/10.1145/3763174>

A Per-Target Results

Tables 2 and 3 give, for each target and arm, the mean cost per cell over the four replicates, mean input/output tokens in thousands (I/O k), the hybrid mean cost as a percentage of agent-only (vs. agent), and whether the specification killed every withheld mutant in all four replicates (✓) or a mutant survived (✗, subscripted with the count where more than one replicate failed); ◦ marks a cell whose scoring stayed inconclusive. Input counts include uncached, cached, and cache-creation tokens.

Table 2. Sol 5.6 by target. Summed over the 20 targets the arms cost \$26.65, \$27.44, and \$28.97.

target	agent-only			hybrid-guided			hybrid-flexible		
	cost	I/O	k ok	cost	vs. agent	I/O k ok	cost	vs. agent	I/O k ok
BA-base-012	\$0.535	449/9.7	✓	\$0.554	104%	398/9.7	\$0.757	141%	677/14.3
BK-bucket-016	\$0.331	329/4.1	✓	\$0.408	123%	403/4.8	\$0.305	92%	257/3.3
LP-price-021	\$0.280	199/3.5	✓	\$0.257	92%	141/3.7	\$0.272	97%	185/4.8
MD-median-015	\$0.152	100/1.5	✓	\$0.191	125%	107/1.6	\$0.195	128%	137/1.5
MM-min-013	\$0.307	208/2.5	✓	\$0.288	94%	219/3.9	\$0.279	91%	189/3.6
OV-order-006	\$0.187	120/2.3	✗ ₄	\$0.190	102%	104/1.3	\$0.233	124%	125/1.6
PM-curve-027	\$0.229	137/2.8	✓	\$0.147	64%	91/1.2	\$0.194	84%	141/1.4
QP-part-025	\$0.618	473/12.7	○	\$0.708	115%	677/13.6	\$0.708	115%	610/12.9
SM-select-022	\$0.450	304/9.2	✓	\$0.787	175%	480/19.0	\$0.473	105%	320/9.9
TL-lev-020	\$0.271	208/3.7	✓	\$0.349	129%	287/5.0	\$0.432	159%	397/5.5
TR-cancel-026	\$0.347	257/5.0	○	\$0.354	102%	294/5.7	\$0.360	104%	266/5.3
TR-discard-011	\$0.299	213/3.5	✓	\$0.329	110%	257/4.5	\$0.360	120%	271/4.1
TR-order-010	\$0.226	172/2.9	✓	\$0.299	132%	262/4.2	\$0.318	141%	297/4.3
TS-trial-019	\$0.362	220/5.3	✓	\$0.236	65%	134/2.8	\$0.234	64%	156/2.3
UC-credits-008	\$0.391	267/4.8	✓	\$0.324	83%	251/4.9	\$0.333	85%	287/4.5
VS-contrib-003	\$0.307	280/3.6	✓	\$0.177	58%	122/1.3	\$0.247	80%	155/1.7
VS-fees-001	\$0.348	276/5.1	✓	\$0.371	107%	305/6.7	\$0.504	145%	422/7.2
VS-redeem-004	\$0.535	446/8.9	✓	\$0.494	92%	497/7.5	\$0.614	115%	635/9.5
VS-shares-002	\$0.274	200/3.0	✓	\$0.171	62%	99/1.6	\$0.222	81%	171/1.9
WU-consume-023	\$0.211	136/2.8	✓	\$0.228	108%	137/3.0	\$0.202	96%	151/2.4
all 20	\$0.333	250/4.9	74/80	\$0.343	103%	263/5.3	\$0.362	109%	292/5.1
						77/80			77/80

Table 3. Terra 5.6 by target. Summed over the 20 targets the arms cost \$18.84, \$15.01, and \$16.60.

target	agent-only			hybrid-guided			hybrid-flexible				
	cost	I/O	ok	cost	vs. agent	I/O	ok	cost	vs. agent	I/O	ok
BA-base-012	\$0.799	1465/26.7	✓	\$0.673	84%	1234/23.8	✓	\$0.753	94%	1481/25.1	✓
BK-bucket-016	\$0.139	197/2.7	✓	\$0.156	112%	247/2.9	✓	\$0.171	122%	264/3.2	✓
LP-price-021	\$0.180	274/4.4	✓	\$0.103	58%	122/2.5	✓	\$0.124	69%	187/2.4	✓
MD-median-015	\$0.123	185/2.6	✓	\$0.076	62%	85/0.9	✓	\$0.107	87%	153/1.4	✓
MM-min-013	\$0.174	258/3.7	✓	\$0.171	98%	269/4.1	✓	\$0.156	89%	268/2.9	✓
OV-order-006	\$0.111	139/2.3	✗ ₂	\$0.081	73%	89/0.7	✓	\$0.092	83%	140/1.0	✓
PM-curve-027	\$0.148	233/3.2	✓	\$0.071	48%	89/0.6	✓	\$0.097	66%	145/1.2	✓
QP-part-025	\$0.289	412/9.1	✗ ₃	\$0.357	124%	537/11.7	✗	\$0.354	122%	617/11.5	✗ ₂
SM-select-022	\$0.315	438/11.0	○	\$0.383	122%	487/15.4	✓	\$0.412	131%	482/17.3	✓
TL-lev-020	\$0.180	273/4.1	✓	\$0.224	125%	376/5.8	✓	\$0.227	126%	383/4.6	✓
TR-cancel-026	\$0.222	334/6.0	✓	\$0.191	86%	266/6.2	✓	\$0.218	98%	320/5.6	✓
TR-discard-011	\$0.159	242/3.8	✓	\$0.188	118%	306/4.4	✓	\$0.178	112%	306/4.1	✓
TR-order-010	\$0.164	228/4.1	✓	\$0.186	113%	263/4.4	✓	\$0.180	110%	286/4.0	✓
TS-trial-019	\$0.152	215/3.9	✓	\$0.073	48%	91/0.9	✓	\$0.080	53%	118/0.9	✓
UC-credits-008	\$0.242	339/6.7	✓	\$0.150	62%	206/3.6	✓	\$0.196	81%	355/4.4	✓
VS-contrib-003	\$0.238	436/4.4	✓	\$0.084	35%	101/0.9	✓	\$0.107	45%	175/1.4	✓
VS-fees-001	\$0.264	418/7.0	✓	\$0.201	76%	272/6.2	✓	\$0.243	92%	395/7.0	✓
VS-redeem-004	\$0.495	893/15.9	✓	\$0.218	44%	386/4.4	✓	\$0.244	49%	479/4.5	✓
VS-shares-002	\$0.175	282/3.0	✓	\$0.089	51%	100/1.1	✓	\$0.101	58%	135/1.1	✓
WU-consume-023	\$0.142	181/3.2	✓	\$0.075	52%	95/0.8	✓	\$0.109	76%	153/1.5	✓
all 20	\$0.236	372/6.4	74/80	\$0.188	80%	281/5.1	79/80	\$0.207	88%	342/5.3	78/80